

Dynamic-Depth Routing for Budget-Adaptive Object Detection in ADAS

HYUNGSEOP LEE¹, JIHO LEE¹, SUNWOO LEE¹, and WOCHUL KANG¹, (Member, IEEE)

¹Department of Embedded Systems Engineering, Incheon National University, Yeonsu-gu 22012, South Korea

Corresponding author: Wochul Kang (wchkang@inu.ac.kr)

This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT). (No. RS-2025-00522938)

ABSTRACT Camera-based object detection in Advanced Driver Assistance Systems (ADAS) frequently encounters dynamically changing computational budgets on resource-constrained edge devices. While recent dynamic-depth neural networks offer flexible computational graphs, they lack a dedicated mechanism for online execution-path selection to maximize accuracy under shifting runtime constraints. In this paper, we propose a feedback-driven dynamic-depth routing framework that decouples the router's training phase from deployment-time budget constraints. By formulating frame-level path selection as a supervised advantage regression problem, our framework employs a lightweight causal router that predicts execution advantage without introducing inference latency by exploiting the temporal coherence of previous-frame features. This router operates in tandem with a feedback controller that dynamically adjusts an execution threshold to satisfy target budgets without requiring network retraining. Extensive evaluations on the KITTI, BDD100K, and Waymo datasets demonstrate that our approach reduces energy consumption by 11–21% while maintaining average precision within 0.1 points of the full-capacity baseline. Furthermore, on-device closed-loop evaluations under continuous scene variations validate that the proposed system reliably tracks time-varying resource budgets, such as target frame rates and target energy consumption, providing a robust and practical solution for real-world edge deployment.

INDEX TERMS Advanced Driver Assistance Systems, 2D Object detection, Dynamic neural networks, Conditional computation, Feedback control, Accuracy–Efficiency Trade-off

I. INTRODUCTION

In Advanced Driver Assistance Systems (ADAS), camera-based object detection is a foundational perception component, supporting Forward Collision Warning [1]–[3], Autonomous Emergency Braking [4], [5], and traffic sign recognition [6], [7]. Because these vision models are typically deployed on resource-constrained edge platforms [8] (e.g., windshield-mounted smart cameras), their available computational budgets fluctuate due to hardware and operational factors such as thermal throttling [9] and power constraints. Moreover, computational demands vary dynamically with driving conditions. Higher vehicle speeds require higher-FPS inference to maintain safe reaction margins, while dense traffic scenes increase the overall perceptual and computational workload. Whereas conventional real-time object detectors [10]–[14] rely on static computational graphs and struggle to maintain an optimal accuracy–efficiency trade-off under these dynamic constraints, dynamic neural net-

works [15]–[18] have emerged as a promising alternative by enabling flexible, adaptive computation during inference.

Recent dynamic-depth neural networks, such as AnyDepth-YOLO/-DETR [18], enable a single detector to operate at multiple computation depths within a unified architecture. This is achieved by decomposing each stage of the network into shared essential layers and modular skippable layers. Executing only the essential layers across all stages forms a lightweight *base-path* for efficient inference, whereas additionally activating the skippable layers yields a full-capacity *super-path* for maximum accuracy. However, while AnyDepth-YOLO/-DETR introduces this flexible architecture, it does not address *how execution paths should be selected online to maximize detection accuracy under runtime budget constraints*.

In this work, we address this problem using a feedback-driven dynamic-depth routing framework, as illustrated in Figure 1. To maximize accuracy under fluctuating budgets, our framework decouples router learning from deployment-

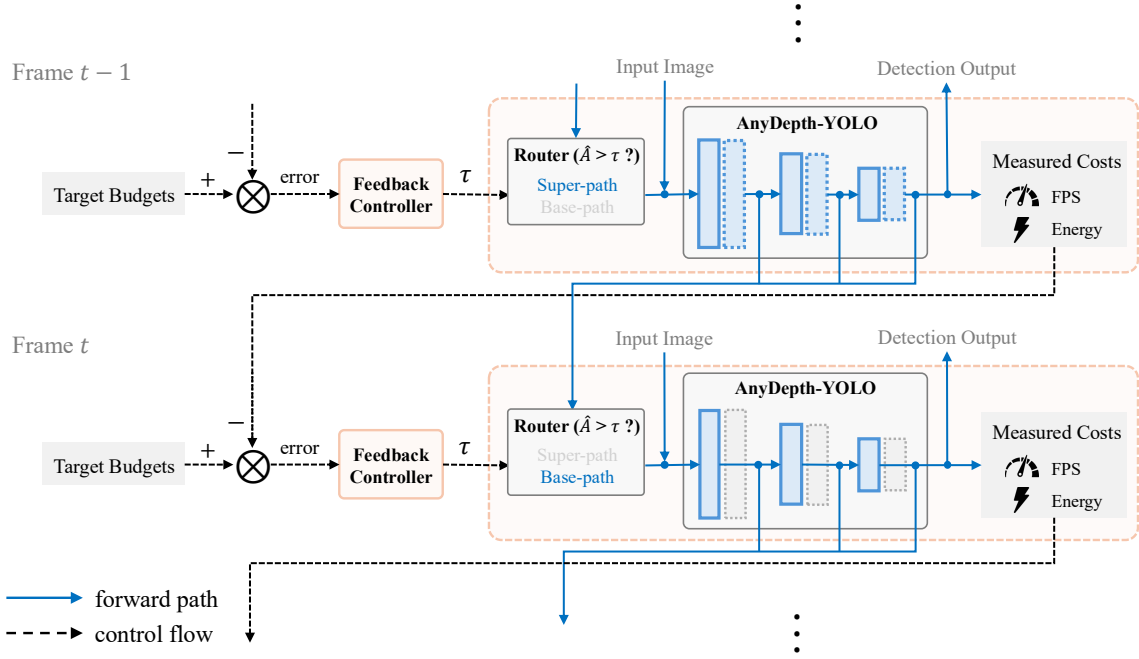


FIGURE 1. Overview of the proposed feedback-driven dynamic routing framework. Exploiting the temporal coherence between adjacent frames in streaming camera inputs, a lightweight router leverages intermediate features from the preceding frame ($t - 1$) to predict the execution advantage ($\hat{A}$) of utilizing the super-path for the current frame (t). If the predicted advantage exceeds a dynamic threshold τ , the current frame is routed to the full-capacity super-path. Otherwise, it is routed to the lightweight base-path. A feedback controller dynamically regulates τ to satisfy varying target resource budgets.

time budget control. During router training, we formulate routing as an advantage regression problem that predicts the accuracy gain of the super-path over the base-path. This continuous score provides the fine-grained signal required for precise, threshold-based runtime control. To eliminate routing overhead during camera-streaming inference, the router reuses features from the previous frame. Since these features may originate from either execution path, we employ path-conditioned training to expose the router to both feature distributions and mitigate the resulting distribution gap. During deployment, a closed-loop feedback controller adjusts the execution threshold according to runtime resource constraints. This enables a single model to continuously navigate the accuracy–efficiency Pareto frontier while satisfying target budgets without retraining.

To validate our approach, extensive evaluations across three representative driving datasets (KITTI, BDD100K, and Waymo) demonstrate that the proposed router reliably maintains the performance of the super-only baseline while trimming redundant computations. Specifically, it reduces per-frame latency and energy consumption by approximately 13% on KITTI, 11% on BDD100K, and 21% on the high-resolution Waymo benchmark, all while maintaining AP within a strictly marginal 0.1 point of the super-only ceiling. Furthermore, integrating the router and AnyDepth-YOLO with a proportional-integral controller enables runtime tracking of time-varying resource targets, such as FPS and energy consumption. Live closed-loop evaluations on an edge device (e.g., NVIDIA Jetson Orin Nano) under continuous scene drifts demonstrate that our system tracks dynamic FPS and

energy budgets with a relative tracking error of less than 3%.

The main contributions of this paper are summarized as follows:

- We propose a dynamic-depth routing framework that formulates execution-path selection as continuous advantage regression. To eliminate inference latency, a lightweight causal router reuses previous-frame features, optimized via path-conditioned training to mitigate cross-path distribution gaps.
- We introduce a closed-loop control scheme that treats the router and detector as the target system to track runtime budgets. By dynamically adjusting the execution threshold, it decouples training from deployment, satisfying time-varying targets without retraining.
- Evaluations on KITTI, BDD100K, and Waymo demonstrate 11% to 21% reductions in latency and energy consumption, maintaining average precision within 0.1 points of the full-capacity baseline. Live on-device deployment confirms precise tracking of dynamic budgets with under 3% relative error.

II. RELATED WORK

A. REAL-TIME OBJECT DETECTION

Recent real-time object detectors continue to push the accuracy–efficiency Pareto frontier through architectural innovations. The YOLO family [10]–[12] integrates advanced mechanisms such as I/O-aware Area Attention [11], [19] and Mixture-of-Experts (MoE) Transformers [12], [20]–[22]. Concurrently, end-to-end architectures like RT-DETR [14], [23], [24] achieve real-time efficiency without heuristic post-

processing (e.g., NMS) by pairing CNN feature extractors with Transformer decoding [25]–[27].

Despite these optimizations, conventional detectors fundamentally rely on static computational graphs. While they offer multiple scale variants (e.g., nano to xlarge) tailored for different hardware platforms, each variant remains a rigid, independently trained model. This lack of architectural flexibility makes runtime adaptation to fluctuating computational budgets inherently impractical.

B. DYNAMIC NEURAL NETWORKS

1) Image Classification

Unlike fixed-graph models, adaptive inference dynamically allocates computation based on input difficulty or runtime constraints. In image classification, early-exit networks [28]–[32] utilize auxiliary classifiers to terminate inference upon reaching high confidence. Alternatively, dynamic layer-skipping [33], [34] bypasses residual blocks per input to eliminate computational redundancy. Furthermore, multi-configuration architectures, such as slimmable networks [35], [36], Once-for-All networks [37], and adaptive-depth networks [16], demonstrate that a single set of weights can span diverse accuracy–efficiency trade-offs, eliminating the need to maintain multiple independent models.

2) Object Detection

Extending adaptive inference to object detection poses unique challenges because modern detectors depend on tightly coupled, multi-scale feature pyramids [38]–[41]. Consequently, applying naive early-exit strategies often disrupts the feature hierarchy, compromising the rich representations required for dense prediction [17], [42], [43]. To preserve this hierarchy without prematurely truncating the network, recent architectures maintain all feature stages and instead dynamically scale their internal computational depth. For instance, DynamicDet [15] cascades two detectors, conditionally executing the second based on input difficulty. More recently, AnyDepth-YOLO/DETR [18] introduced a unified multi-depth detector by decomposing each stage into essential and skippable layers. Through a self-distillation training strategy that enforces stage-wise modularity, this architecture enables a single model to span a continuous accuracy–efficiency spectrum while preserving the full feature pyramid.

However, AnyDepth-YOLO/DETR focuses exclusively on the network architecture and training paradigm, without addressing how execution paths should be selected online under fluctuating resource budgets. We build directly upon this foundation to address the challenge of dynamic routing during deployment.

III. PRELIMINARIES: ANY-DEPTH YOLO

Our framework builds upon AnyDepth-YOLO [18], a dynamic-depth object detector that adjusts its execution depth at inference time without retraining. To preserve the multi-scale feature hierarchy, it divides each backbone and neck stage into a *shared essential layer* (executed for core features)

TABLE 1. Accuracy–efficiency trade-offs of the base and super execution paths for AnyDepth-YOLO (S) [18] on the BDD100K detection validation set (720×1280 input resolution) [44]. FPS and energy consumption are measured on an NVIDIA Jetson Orin Nano using TensorRT FP16.

Path	#Params	GFLOPs	AP	FPS	Energy (mJ)
Super	7.42M	51.72	35.0	24	866
Base		33.18	34.1	40	512

and a *skippable layer* (executed for feature refinement). To jointly optimize both layers within a single set of parameters, AnyDepth-YOLO employs a self-distillation training strategy that encourages consistent feature representations between their outputs. This yields stage-wise modularity, enabling arbitrary combinations of execution depths while preserving network compatibility.

Depending on the combination of executed skippable layers, AnyDepth-YOLO provides a wide spectrum of intermediate operating points. At the two extremes of this configuration space are the full-capacity super-path δ^{super} , which activates all layers, and the lightweight base-path δ^{base} , which executes only the essential layers. As summarized in Table 1, since they operate within a single model, both paths share the same parameter space but offer distinct accuracy–efficiency trade-offs. Specifically, by executing only the essential layers, the base-path achieves a 66% throughput increase and a 41% energy reduction at the cost of a 0.9 AP degradation, serving as an ideal fallback under constrained budgets.

IV. METHOD

Our framework dynamically routes each video frame through either the super- or base-path of AnyDepth-YOLO to maximize detection accuracy under fluctuating computational budgets. Although the detector supports multiple intermediate operating points, we adopt binary routing between the two extreme paths. Since deployment objectives such as FPS and energy consumption are typically evaluated over temporal windows rather than strict per-frame constraints, this strategy provides sufficient budget control while simplifying the routing decision space.

We realize this framework by explicitly decoupling the system into two distinct phases. First, the training phase (Section IV-A) formulates path selection as an advantage regression problem, optimizing a lightweight router entirely independent of deployment-time resource constraints. Second, the deployment phase (Section IV-B) utilizes this trained router for causal online inference. The router evaluates the previous frame’s intermediate features to predict the current frame’s advantage score, while a feedback controller dynamically adjusts the routing threshold to satisfy the target budget.

A. TRAINING PHASE: ADVANTAGE REGRESSION FOR DYNAMIC ROUTING

Figure 2 illustrates the overall training pipeline of our proposed advantage-based router. While the AnyDepth-YOLO remains entirely frozen, the lightweight router is optimized to

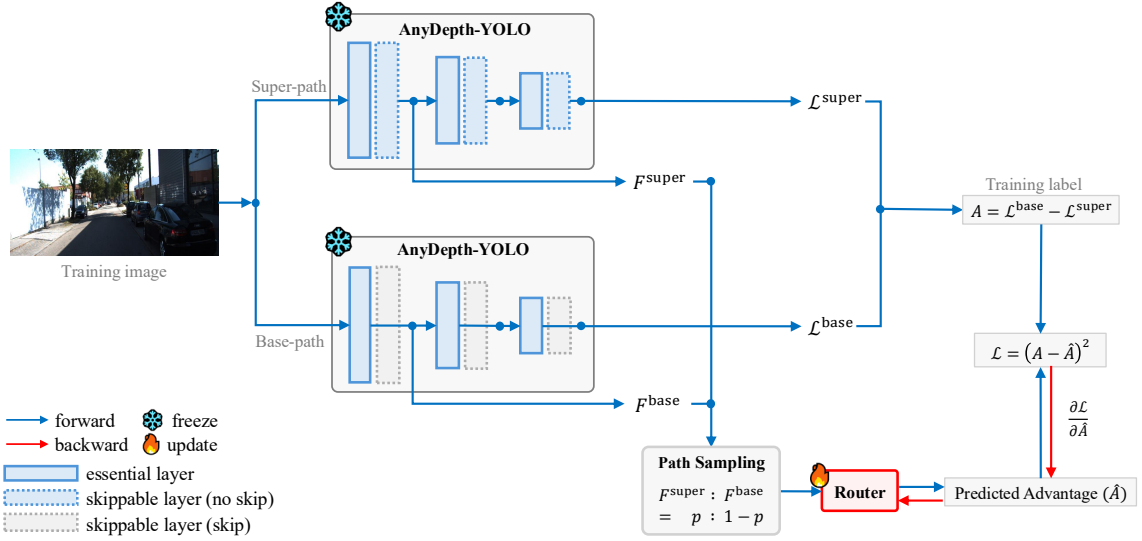


FIGURE 2. Training pipeline of the proposed advantage-based router. The backbone detector (AnyDepth-YOLO) remains frozen, while only the lightweight router is optimized. The ground-truth advantage is computed as $A = \mathcal{L}^{\text{base}} - \mathcal{L}^{\text{super}}$ using the outputs of both execution paths. To improve robustness to varying runtime paths, the router is trained with probabilistic path sampling ($p : 1 - p$) and optimized using the regression loss $(A - \hat{A})^2$.

regress the continuous advantage score. The specific components of this training phase, including the router architecture, training objective, and path-conditioned training strategy, are detailed in the following subsections.

1) Router Architecture

Because the router operates on a per-frame basis during deployment, minimizing its computational overhead is essential. Formally, the router R maps an intermediate multi-scale feature set F to a continuous advantage score:

$$\hat{A} = R(F). \quad (1)$$

Depending on the active execution path, the frozen detector yields a feature set $F \in \{F^{\text{super}}, F^{\text{base}}\}$. This set comprises L_B backbone stages $\{B_l\}_{l=1}^{L_B}$ and L_N neck stages $\{N_l\}_{l=1}^{L_N}$, where $B_l, N_l \in \mathbb{R}^{C_l \times H_l \times W_l}$. To compute $R(F)$, the router processes these inputs in three sequential steps.

First, to extract a lightweight descriptor for each feature group $X \in \{B, N\}$, the multi-stage features are pooled to a common $G \times G$ grid to align their spatial resolutions. The pooled maps are then concatenated channel-wise:

$$\tilde{z}^X = [\text{AvgPool}_{G \times G}(X_1), \dots, \text{AvgPool}_{G \times G}(X_{L_X})] \in \mathbb{R}^{C \times G \times G}, \quad (2)$$

where $C = \sum_l C_l$.

Second, a 1×1 convolution followed by batch normalization and ReLU projects $\tilde{z}^X$ into a uniform channel dimension d . A 3×3 depthwise convolution [45] then mixes the spatial information, followed by global average pooling to yield the group-level descriptor:

$$z^X = \text{GAP}\left(\text{DWConv}\left(\text{ReLU}\left(\text{BN}\left(\text{Conv}_{1 \times 1}(\tilde{z}^X)\right)\right)\right)\right) \in \mathbb{R}^d. \quad (3)$$

Finally, the backbone and neck descriptors are concatenated into a joint vector $z = [z^B, z^N] \in \mathbb{R}^{2d}$ and passed

TABLE 2. Per-frame router overhead. Grid denotes the spatial pooling resolution ($G \times G$), and Feature indicates the input source (backbone, neck, or both). This fixed-grid pooling ensures the cost remains independent of input resolution. All variants add negligible overhead compared to the super-path (26.30 GFLOPs at 384×1248); our default setting (2×2 , both) consumes $< 0.003\%$ of this total.

Grid	Feature	#Params	GFLOPs	% of SUPER
1×1	backbone	60,241	1.17×10^{-4}	0.0004
	neck	51,793	1.01×10^{-4}	0.0004
	both	110,737	2.15×10^{-4}	0.0008
2×2	backbone	60,881	4.17×10^{-4}	0.0016
	neck	52,433	3.51×10^{-4}	0.0013
	both (default)	112,017	7.65×10^{-4}	0.0029
8×8	backbone	60,881	6.38×10^{-3}	0.0243
	neck	52,433	5.34×10^{-3}	0.0203
	both	112,017	1.17×10^{-2}	0.0446

through two fully connected (FC) layers with a hidden dimension h to output the advantage score:

$$\hat{A} = \text{FC}_{h \rightarrow 1}(\text{ReLU}(\text{FC}_{2d \rightarrow h}(z))). \quad (4)$$

By leveraging aggressive spatial pooling and lightweight depthwise convolution, the routing mechanism introduces negligible computational overhead. As reported in Table 2, our default setting consumes less than 0.003% of the super-path's total GFLOPs. A comprehensive ablation study justifying these router design choices is detailed in Section V-E.

2) Training Objective

By formulating path selection as an advantage prediction task, the router is directly optimized using a standard mean squared error (MSE) loss:

$$\mathcal{L}_{\text{router}} = (A - \hat{A})^2, \quad (5)$$

where $\hat{A}$ represents the advantage score predicted by the router, and A denotes the ground-truth advantage, defined as the reduction in detection loss achieved by executing the super-path rather than the base-path:

$$A = \mathcal{L}_{\text{base}} - \mathcal{L}_{\text{super}}. \quad (6)$$

Here, $\mathcal{L}_{\text{super}}$ and $\mathcal{L}_{\text{base}}$ denote the detection losses associated with the super and base configurations, respectively.

Crucially, this regression-based formulation decouples the training objective from runtime resource constraints, enabling a single trained model to navigate the accuracy-efficiency trade-off during deployment without budget-specific retraining. Furthermore, because AnyDepth-YOLO remains frozen during router optimization, both the ground-truth targets A and intermediate features F can be pre-computed offline. This offline extraction eliminates redundant forward passes through the detector during training, ensuring a highly efficient optimization process for the lightweight router.

3) Path-Conditioned Training

During real-time deployment on video streams, the router exploits temporal coherence by causally reusing the intermediate feature map from the preceding frame. This avoids the computational bottleneck of extracting current-frame features prior to routing. However, because the system dynamically switches execution paths, the source of this feature map varies between F^{super} and F^{base} depending on the prior routing decision.

To ensure the router remains robust to both feature distributions during optimization, we introduce a stochastic feature sampling strategy. Formally, during training, the input feature set F provided to the router R is sampled conditionally based on a Bernoulli random variable $M \sim \text{Bernoulli}(p)$:

$$F = \begin{cases} F^{\text{super}}, & \text{if } M = 1, \\ F^{\text{base}}, & \text{if } M = 0, \end{cases} \quad (7)$$

where p represents the probability of sampling from the super-path. Consequently, the router is optimized to minimize the expected loss over this path-conditioned input distribution, reformulating Equation (5) as:

$$\mathcal{L}_{\text{router}} = \mathbb{E}_{M \sim \text{Bernoulli}(p)} \left[(A - R(F))^2 \right]. \quad (8)$$

This stochastic sampling prevents the router from overfitting to the feature distribution of a single path, forcing it to maintain consistent accuracy regardless of which path generated the input representations. As demonstrated in Section V-E3, this strategy is essential for mitigating distribution shifts during robust runtime routing.

B. DEPLOYMENT PHASE: FEEDBACK-DRIVEN BUDGET ADAPTATION

During deployment, the entire inference pipeline, comprising the router and the AnyDepth-YOLO detector, is treated as the target system under feedback control. To regulate its resource consumption, the controller dynamically adjusts a

Algorithm 1 Causal Budget-Adaptive Depth Routing via Feedback Control

Input: frozen AnyDepth detector D , router R , target budget sequence $C_{\text{target}}(t)$,

PI gains k_p, k_i , EMA factor α , operational bounds

$\tau_{\min}, \tau_{\max}$

Output: streaming per-frame detections y_t

```

1  config  $\leftarrow \delta^{\text{base}}$ 
2   $\tau \leftarrow \tau_0, \bar{C} \leftarrow C_{\text{target}}(1), I \leftarrow 0$ 
3  for  $t = 1$  to  $T$  do:
4     $(y_t, F_t) \leftarrow D(x_t, \text{config})$ 
5    emit  $y_t$ 
6     $\hat{A} \leftarrow R(F_t)$ 
7    config  $\leftarrow \delta^{\text{super}}$  if  $\hat{A} > \tau$  else  $\delta^{\text{base}}$   $\triangleright$  route next frame
8     $C \leftarrow \text{MeasureResourceCost}()$ 
9     $\bar{C} \leftarrow (1 - \alpha)\bar{C} + \alpha C$ 
10    $e \leftarrow C_{\text{target}}(t) - \bar{C}$ 
11    $\tau_{\text{ff}} \leftarrow \text{InterpolateProfiles}(C_{\text{target}}(t))$ 
12    $\tau_{\text{un}} \leftarrow \tau_{\text{ff}} - k_p e - k_i(I + e)$   $\triangleright$  PI control with feedforward
13    $\tau \leftarrow \text{clip}(\tau_{\text{un}}, \tau_{\min}, \tau_{\max})$ 
14    $I \leftarrow I + e + \frac{\tau_{\text{un}} - \tau}{k_i}$   $\triangleright$  anti-windup back-calculation
15 end for

```

single continuous control variable, the routing threshold $\tau(t)$, on a per-frame basis.

To bypass the latency overhead of extracting current-frame features prior to routing, we employ a causal routing mechanism, where the execution path for frame t , $\text{config}(t) \in \{\delta^{\text{super}}, \delta^{\text{base}}\}$, is determined by the advantage score predicted from the previous frame's intermediate features, F_{t-1} :

$$\text{config}(t) = \begin{cases} \delta^{\text{super}}, & \text{if } \hat{A}(F_{t-1}) > \tau(t), \\ \delta^{\text{base}}, & \text{otherwise.} \end{cases} \quad (9)$$

Increasing $\tau(t)$ biases routing toward the base-path to conserve computational resources, whereas decreasing it favors the super-path to improve detection accuracy.

To satisfy dynamic resource budgets without retraining the detector, we employ a proportional–integral (PI) controller. Let $C_{\text{target}}(t)$ and $C(t)$ denote the target budget and the measured cost at frame t , respectively¹. Since routing decisions are binary, the instantaneous resource consumption exhibits high frame-to-frame variance, which can destabilize the control loop. To mitigate this effect, we first apply an exponential moving average (EMA) filter to obtain a smoothed estimate of resource consumption:

$$\bar{C}(t) = (1 - \alpha)\bar{C}(t-1) + \alpha C(t), \quad (10)$$

where $\alpha \in (0, 1]$ is the smoothing factor. Although the EMA introduces a slight phase delay, it effectively suppresses instantaneous frame-to-frame fluctuations while preserving the time-averaged consumption trend.

The tracking error $e(t)$ is computed as

$$e(t) = C_{\text{target}}(t) - \bar{C}(t). \quad (11)$$

¹For throughput metrics such as FPS, the target and measured values are internally converted to latency so that lower values consistently represent reduced resource usage, ensuring $e(t) < 0$ always indicates over-consumption.

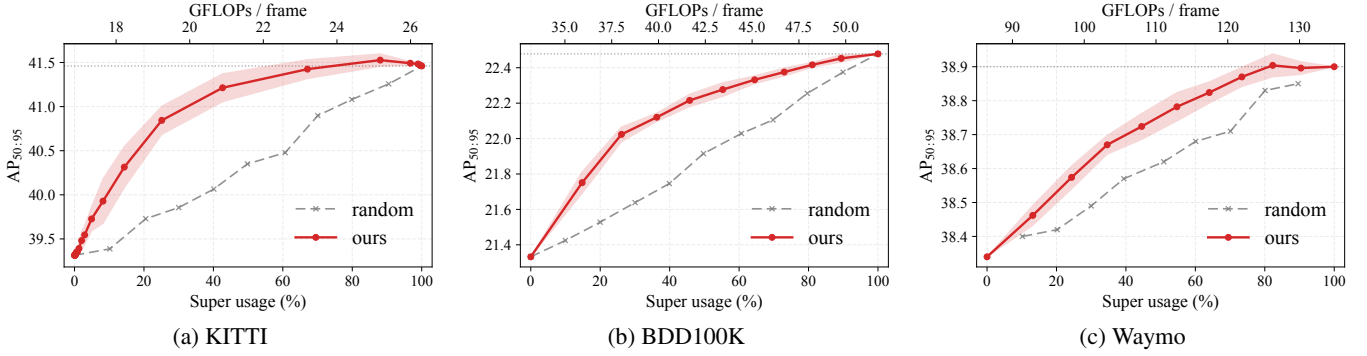


FIGURE 3. Accuracy-efficiency Pareto frontiers of dynamic depth routing on (a) KITTI MOT, (b) BDD100K MOT, and (c) Waymo Open. We evaluate our advantage-based router against a random routing baseline in terms of $AP_{50:95}$ versus super-path usage ratio. Each operating point is obtained by simply adjusting the routing threshold τ , without retraining. A single trained router establishes a continuous Pareto frontier that consistently outperforms the baseline across all three benchmarks.

The unclipped threshold $\tau_{un}(t)$ is updated via the positional PI control law:

$$\tau_{un}(t) = \tau_{ff}(t) - k_p e(t) - k_i (I(t-1) + e(t)), \quad (12)$$

where $\tau_{ff}(t)$ is a feedforward term obtained by linearly interpolating the offline cost profiles from Table 3 to map $C_{target}(t)$ to the corresponding threshold, $I(t-1)$ is the accumulated error, and k_p, k_i are the control gains. To prevent integral windup, we clamp $\tau(t)$ to $[\tau_{min}, \tau_{max}]$ and adjust the accumulated error $I(t)$ via back-calculation:

$$\begin{aligned} \tau(t) &= \text{clip}(\tau_{un}(t), \tau_{min}, \tau_{max}), \\ I(t) &= I(t-1) + e(t) + \frac{\tau_{un}(t) - \tau(t)}{k_i}. \end{aligned} \quad (13)$$

Detailed configurations for the controller parameters are provided in Section V-D.

V. EXPERIMENT

A. EXPERIMENTAL SETUP

1) Datasets

We evaluate our approach on three representative autonomous driving benchmarks. Across all datasets, the router is trained exclusively on static images and evaluated on continuous video sequences, without requiring any temporal or tracking supervision at any stage. This setup explicitly demonstrates the router's ability to generalize from independent static frames to temporally correlated video streams during deployment.

KITTI. We train the router on the KITTI Detection dataset [46], utilizing an 80:20 split of 5,985 training and 1,496 validation images. Evaluation is performed on the KITTI Multi-Object Tracking (MOT) benchmark, which comprises 21 continuous video sequences (8,008 frames). Input frames are processed at a resolution of 384×1248 .

BDD100K. To evaluate our framework on a highly diverse driving dataset, we train the router on the BDD100K detection set [44] (70K training, 10K validation images) and evaluate it on the BDD100K MOT validation split, which contains 200 video sequences. Input frames are processed at a resolution of 720×1280 .

Waymo. To demonstrate scalability to a large-scale, high-resolution benchmark, we utilize front-camera data from the Waymo Open Perception dataset [47]. Because this benchmark does not provide separate detection and tracking subsets, we repurpose the standard official splits: the router is trained on 158.1K frames from the 798 training segments, processed as independent static images. It is then evaluated on the 202 validation segments (39.1K labeled frames) processed as continuous video. Input frames are processed at a resolution of 1280×1920 .

2) Implementation Details

The router is trained for 30 epochs using Adam optimizer [48] (batch size 256, learning rate 10^{-3}). Results are averaged across five random seeds (0–4) and reported as mean $\pm$ standard deviation. Since the router is trained offline on pre-computed features, image-space augmentation is not applied. We also omit additional regularization techniques, including weight decay and dropout.

For continuous video evaluation, we apply a 0.25 confidence threshold and a 0.7 NMS IoU threshold. We report the mean Average Precision ($AP_{[.50:.95]}$) following the COCO protocol [49], the average super-path execution rate, and the theoretical computational cost (GFLOPs per frame). To validate edge deployment practicality, on-device efficiency metrics (per-frame latency, FPS, and energy consumption) are measured directly on an NVIDIA Jetson Orin Nano using TensorRT [50] FP16 engines.

B. EVALUATION OF ADVANTAGE-BASED ROUTING

To validate the effectiveness of the proposed advantage-based routing, we evaluate its ability to balance the accuracy-efficiency trade-off. Because this work introduces the first frame-level routing mechanism tailored for a unified multi-depth detector, there are no directly equivalent prior methods. Consequently, we establish a random routing strategy as our primary baseline, which allocates frames to the super-path at a uniform random ratio to serve as a naive lower bound. As illustrated in Figure 3, a single trained router traces a continuous Pareto frontier that dominates all baselines across

TABLE 3. Efficiency trade-offs of threshold-based routing across the operating points in Figure 3. Latency, FPS, and energy consumption are measured on an NVIDIA Jetson Orin Nano (TensorRT FP16) and include router overhead. *always-super* and *always-base* denote the static single-path baselines corresponding to the two endpoints of the accuracy–efficiency trade-off. The monotonic relationship between the routing threshold τ and resource consumption provides an effective control variable for our feedback-driven budget adaptation.

(a) KITTI							(b) BDD100K						
τ	Super (%)	GFLOPs	AP	Latency (ms)	FPS	Energy (mJ)	τ	Super (%)	GFLOPs	AP	Latency (ms)	FPS	Energy (mJ)
<i>always-base</i>	0.0	16.87	39.3	12.86	77.8	246	<i>always-base</i>	0.0	33.18	21.3	24.99	40.0	512
+0.70	2.9	17.14	39.5	13.10	76.4	250	+0.086	14.8	35.92	21.8	27.46	36.4	565
+0.50	8.2	17.64	39.9	13.54	73.8	259	+0.072	26.1	38.02	22.0	29.33	34.1	605
+0.40	14.3	18.22	40.3	14.04	71.2	269	+0.064	36.3	39.91	22.1	31.03	32.2	641
+0.30	25.1	19.24	40.8	14.94	66.9	286	+0.058	45.8	41.66	22.2	32.61	30.7	674
+0.20	42.7	20.89	41.2	16.39	61.0	315	+0.054	55.3	43.43	22.3	34.20	29.2	708
+0.10	67.1	23.20	41.4	18.41	54.3	355	+0.049	64.5	45.14	22.3	35.73	28.0	741
+0.00	88.1	25.18	41.5	20.15	49.6	389	+0.045	73.0	46.72	22.4	37.14	26.9	771
−0.10	96.7	25.99	41.5	20.85	48.0	403	+0.040	81.1	48.21	22.4	38.49	26.0	799
−0.40	99.6	26.27	41.5	21.09	47.4	408	+0.033	89.4	49.76	22.5	39.88	25.1	829
<i>always-super</i>	100.0	26.30	41.5	21.13	47.3	408	<i>always-super</i>	100.0	51.72	22.5	41.64	24.0	866

(c) Waymo						
τ	Super (%)	GFLOPs	AP	Latency (ms)	FPS	Energy (mJ)
<i>always-base</i>	0.0	86.46	38.3	103.51	9.7	2265
+0.152	13.2	92.84	38.5	114.79	8.7	2513
+0.116	24.4	98.25	38.6	124.36	8.0	2723
+0.098	34.6	103.21	38.7	133.08	7.5	2914
+0.085	44.5	108.00	38.7	141.53	7.1	3100
+0.074	54.7	112.92	38.8	150.25	6.7	3291
+0.066	64.0	117.43	38.8	158.20	6.3	3466
+0.057	73.4	121.96	38.9	166.23	6.0	3642
+0.048	82.3	126.27	38.9	173.83	5.8	3809
+0.039	90.4	130.19	38.9	180.76	5.5	3961
<i>always-super</i>	100.0	134.84	38.9	188.96	5.3	4141

all benchmarks. Table 3 details the operating points along this curve. By adjusting a single routing threshold, τ , the system smoothly traverses the spectrum from the highly efficient *always-base* to the highly accurate *always-super* configuration, dynamically trading off latency, FPS, and energy consumption without requiring model retraining.

Specifically, on the KITTI dataset (Figure 3(a), Table 3(a)), the router maintains an AP@[.50:.95] within 0.1 points of the *always-super* ceiling (41.4 vs. 41.5) while routing only 67% of frames through the super path ($\tau = +0.10$). This targeted allocation reduces theoretical computation from 26.30 to 23.20 GFLOPs/frame (an 11.8% decrease), measured latency from 21.13 to 18.41 ms (12.9% decrease), and energy consumption from 408 to 355 mJ (13.0% decrease), while increasing throughput from 47.3 to 54.3 FPS (a 14.8% gain).

This robust trend holds on BDD100K (Figure 3(b), Table 3(b)). Similarly maintaining accuracy within 0.1 points of the ceiling (22.4 vs. 22.5 AP@[.50:.95]), the router effectively limits super-path execution to 73.0% of the frames ($\tau = +0.045$). Consequently, it reduces computation from 51.72 to 46.72 GFLOPs (9.7% decrease), latency from 41.64 to 37.14 ms (10.8% decrease), and energy usage from 866 to 771 mJ (11.0% decrease), yielding a throughput increase from 24.0 to 26.9 FPS (a 12.1% gain).

Furthermore, this reliable routing behavior scales to the high-resolution Waymo Open benchmark (Figure 3(c), Table 3(c)). At a 54.7% super-path execution rate ($\tau = +0.074$),

the router again performs within 0.1 points of the accuracy ceiling (38.8 vs. 38.9 AP@[.50:.95]). This yields significant hardware efficiency improvements, including a 16.3% reduction in computation (134.84 to 112.92 GFLOPs), a 20.5% decrease in latency (188.96 to 150.25 ms), and a 20.5% drop in energy consumption (4141 to 3291 mJ), alongside a 26.4% increase in throughput (5.3 to 6.7 FPS).

C. ANALYSIS OF ROUTING QUALITY

Since deployment-time routing is performed by thresholding the predicted advantage $\hat{A}$, it is sufficient for the router to rank frames according to their expected benefit from deeper execution, rather than precisely estimating the advantage magnitude. To evaluate this property, we sort the validation images by $\hat{A}$ and partition them into ten equally sized rank bins to compute the mean true advantage for each bin, as shown in Figure 4. Across all three datasets, although minor fluctuations exist between adjacent bins, the mean true advantage exhibits a general increasing trend with the predicted rank. This indicates that higher-ranked frames consistently benefit more from the super-path, demonstrating that the router provides a reliable ranking signal for threshold-based routing.

The pronounced increase in the highest-ranked bin (Figure 4) is consistent with the right-skewed advantage distributions shown in Figure 5. For all three datasets, most frames yield near-zero advantages, while a small fraction

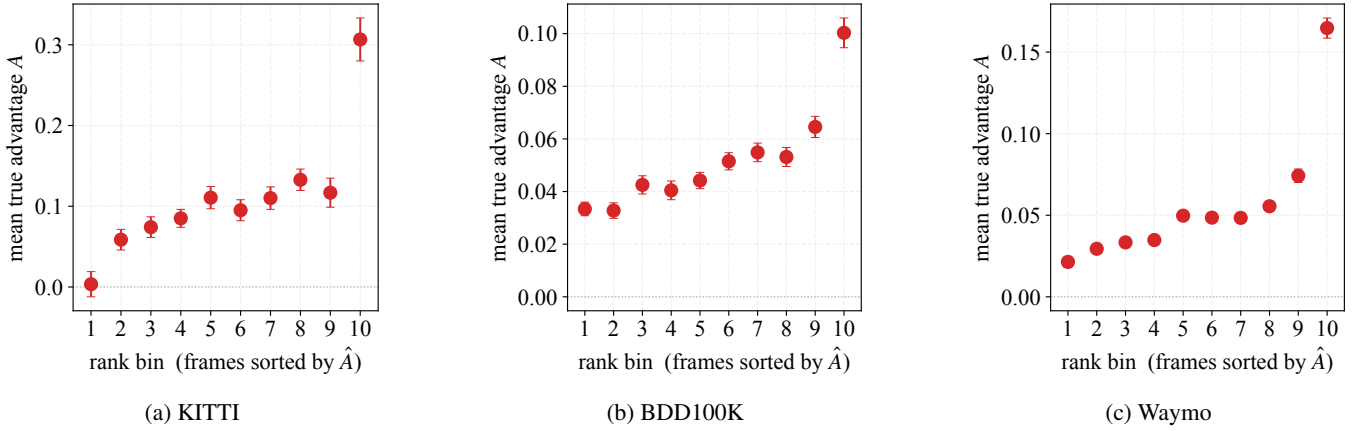


FIGURE 4. Ranking reliability of the router across the validation sets. Validation images are sorted by the predicted advantage $\hat{A}$ (5-seed ensemble) and partitioned into 10 equally sized rank bins. Each marker denotes the mean true advantage $A = \mathcal{L}_{\text{base}} - \mathcal{L}_{\text{super}}$ within each bin; error bars indicate the standard error of the mean. The mean true advantage shows an increasing trend across the rank bins, indicating that higher predicted advantages generally correspond to greater benefits from deeper execution.

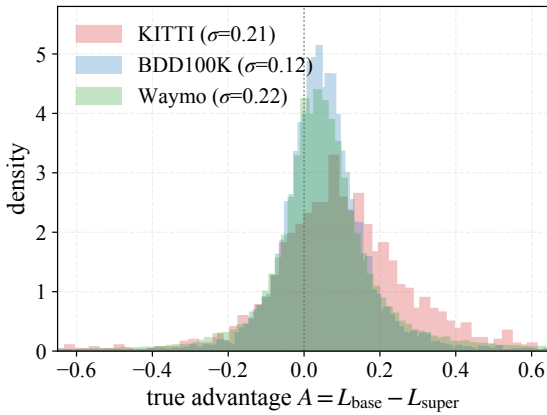


FIGURE 5. Distribution of per-image advantages $A = \mathcal{L}_{\text{base}} - \mathcal{L}_{\text{super}}$ across the validation sets. The distributions are strongly right-skewed, with most samples concentrated near zero. This indicates that deeper execution yields only modest gains for most inputs while providing substantial improvements for a small subset of high-payoff frames.

forms a long positive tail of frames with substantial execution advantages. The router concentrates these rare frames into the top bin, where additional computation maximizes returns. Interestingly, BDD100K exhibits a substantially smaller advantage variance ($\sigma = 0.12$) than KITTI ($\sigma = 0.21$) and Waymo ($\sigma = 0.22$). This reflects an inherent characteristic of AnyDepth-YOLO, where the performance gap between the base-path and the super-path is fundamentally narrower on BDD100K. As a result, the mean true advantage in Figure 4(b) spans a much narrower scale than the wider dispersion seen in Figures 4(a) and (c). This highlights that even when the intrinsic benefits of deeper execution are marginal, our router remains effective at prioritizing the most valuable frames.

D. RUNTIME RESOURCE-BUDGET TRACKING

We evaluate the proposed framework under time-varying resource constraints to assess its closed-loop budget adapta-

tion capability. Specifically, we define the combination of our router and AnyDepth-YOLO as the target system, and couple it with a standard PI controller to track dynamically changing FPS and energy budgets in real time. We then deploy this setup on an NVIDIA Jetson Orin Nano to process BDD100K MOT sequences, enabling us to observe closed-loop adaptation under realistic non-stationary conditions. These sequences contain frequent transitions between diverse driving scenarios (e.g., night $\leftrightarrow$ daytime, city $\leftrightarrow$ highway, and clear $\leftrightarrow$ rainy), resulting in continuously varying computational demands. Because routing decisions are binary, the raw instantaneous cost exhibits severe frame-to-frame chattering. Plotting these raw values directly results in dense visual noise, making it difficult to visually verify the tracking performance. To clearly illustrate the underlying tracking trend, Figure 6 visualizes the measured costs using an offline zero-phase moving average over a short window (30 frames).

As illustrated in Figure 6, the controller effectively adjusts the routing threshold τ to track both step and sawtooth budget profiles despite continuous scene variations. For FPS control (Figure 6(a)), the system closely follows the target frame rate, achieving a Mean Absolute Error (MAE) of 0.38–0.70 FPS. Likewise, for energy consumption control (Figure 6(b)), it accurately tracks the target energy budget with an MAE of 6.1–10.2 mJ. Overall, these results indicate that the routing mechanism remains robust against continuous scene transitions. Although execution decisions are strictly binary on a frame-by-frame basis, continuously adjusting the routing threshold allows for stable budget tracking over time, effectively mitigating the impact of discrete frame-level switching.

The primary objective of this experiment is to show that the combined router and AnyDepth-YOLO, acting as the target system, can effectively regulate execution costs under a feedback loop. Because our focus is on the system's practical controllability rather than rigorous controller analysis or optimization, we simply adopted the empirical parameter configuration summarized in Table 4 for the results in Fig-

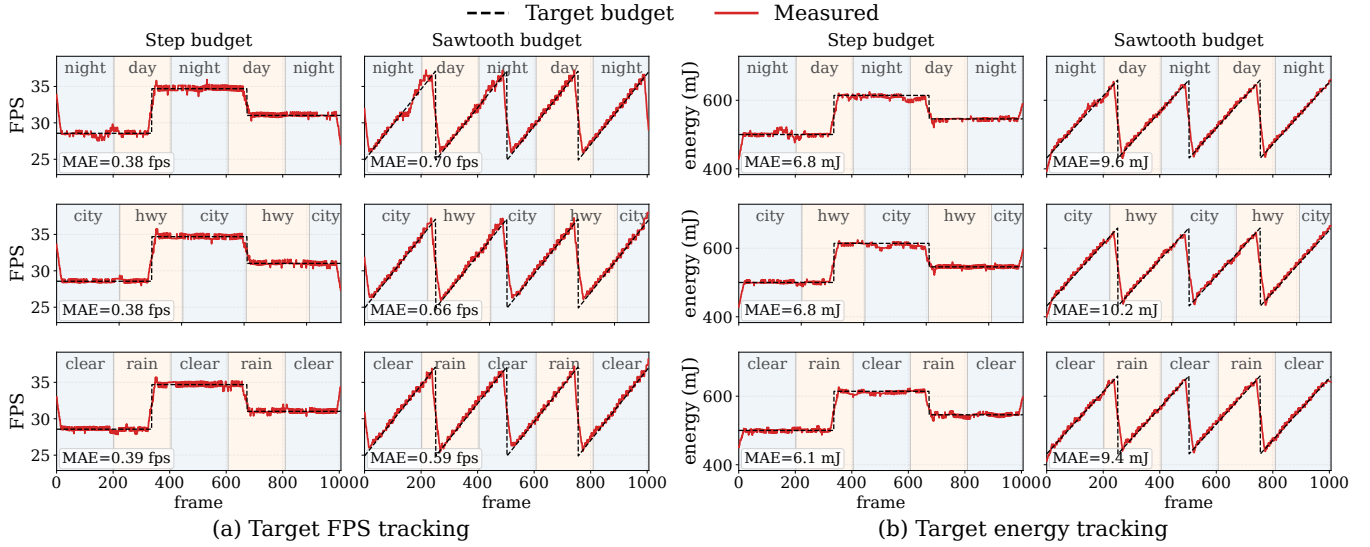


FIGURE 6. Runtime resource-budget tracking on BDD100K MOT (Jetson Orin Nano, 720×1280). Shaded regions denote alternating scene conditions. A PI controller dynamically adjusts the routing threshold τ to track step and sawtooth budget profiles (black dashed lines). The measured metrics (red solid lines) are shown after applying a zero-phase moving average over a short temporal window (30 frames), as the raw measurements are inherently dominated by high-frequency fluctuations arising from binary routing, which obscure the underlying budget-tracking behavior. The system successfully tracks (a) target FPS with an MAE of 0.38–0.70 FPS and (b) target energy consumption with an MAE of 6.1–10.2 mJ.

TABLE 4. Tracking performance under various PI controller configurations on BDD100K MOT (720×1280 , Jetson Orin Nano). Rise, Settling, OS, and SSE denote rise time, settling time, overshoot, and steady-state tracking error, respectively. The row marked * corresponds to the empirical configuration used for the runtime tracking experiments in Figure 6. N/A indicates that metrics are unmeasurable due to severe system chattering/instability.

Parameters			Control Metrics			
α	k_p	k_i	Rise (f) (Up / Dn)	Settling (f) (Up / Dn)	OS (%) (Up / Dn)	SSE (ms) (Step/Saw)
0.83	2.0	0.33	17 / 10	15 / 8	14.2 / 39.4	0.41 / 0.59
0.85*	2.0	0.33	17 / 8	15 / 5	14.1 / 43.5	0.40 / 0.57
0.93	2.0	0.33	13 / 6	29 / 26	53.6 / 83.1	0.45 / 0.97
0.85	0.5	0.33	11 / 7	12 / 13	31.5 / 48.0	0.42 / 0.63
0.85	5.0	0.33	76 / 4	85 / 3	N/A / 35.8	1.36 / 1.58
0.85	2.0	0.10	38 / 9	30 / 4	11.1 / 49.2	0.55 / 0.86
0.85	2.0	1.00	14 / 10	12 / 7	18.1 / 39.3	0.32 / 0.55

ure 6. Specifically, the EMA factor and PI gains were set to $\alpha = 0.85$, $k_p = 2.0$, and $k_i = 0.33$, respectively, based on practical control considerations including rise time, settling time, overshoot, and steady-state tracking error.

E. ABLATION ON ROUTER ARCHITECTURE AND TRAINING STRATEGY

To validate the key design choices of the proposed routing framework, we ablate (1) the router feature source, (2) the router architecture and pooling grid resolution, and (3) the previous-action sampling ratio p on the KITTI and BDD100K datasets, which exhibit distinct scene characteristics and data distributions. Unless otherwise specified, the optimal configuration from each study is adopted as the default.

1) Router Feature Source

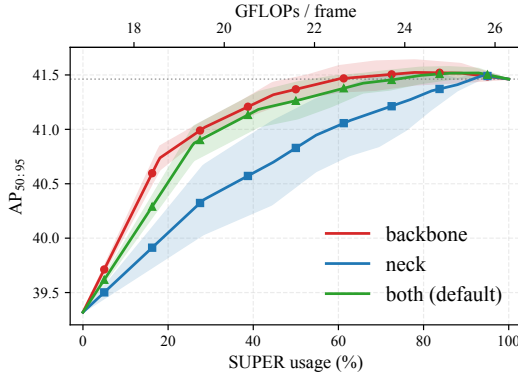
We compare three router feature sources: backbone features, neck features, and their concatenation. While the backbone extracts hierarchical visual representations from the input image, the neck integrates multi-scale semantics through feature fusion. Given these distinct levels of abstraction, we investigate whether a single feature source is sufficient or if their combination provides a more robust representation.

As shown in Figure 7, the effectiveness of a single feature source depends strongly on the dataset. On KITTI (Figure 7(a)), the neck-only router exhibits lower accuracy and higher variance, whereas the backbone-only router performs competitively. On BDD100K (Figure 7(b)), the opposite trend is observed, with the backbone-only router showing the largest variance. This behavior indicates that neither feature source alone consistently captures frame difficulty across diverse scenes. Backbone features lack fully aggregated task-specific semantics, whereas neck features are less sensitive to low-level appearance cues that remain informative in relatively simple scenes such as KITTI. In contrast, combining both feature sources consistently achieves comparable mean performance with the lowest variance across both datasets, confirming their complementary nature. Since the additional computational overhead is negligible, as reported in Table 2, the combined feature source is adopted throughout the paper.

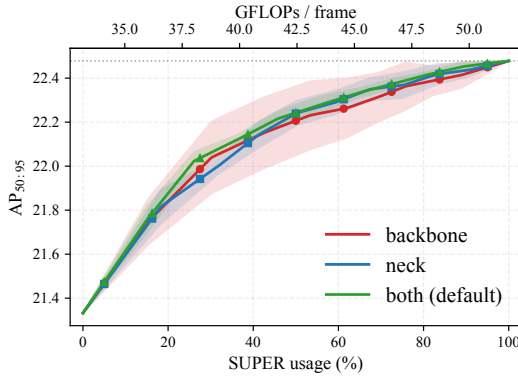
2) Pooling Grid Size

We study the effect of the pooling grid size $G \times G$ used to aggregate multi-scale detector features (Equation 2). Specifically, we compare a spatially agnostic router (1×1) with coarse (2×2) and fine-grained (8×8) grids.

As illustrated in Figure 8, the performance of the grid reso-



(a) KITTI



(b) BDD100K

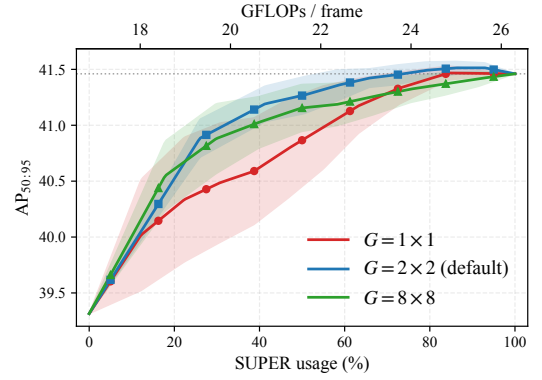
FIGURE 7. Ablation on router feature sources on (a) KITTI and (b) BDD100K. While individual sources exhibit high variance depending on the dataset (neck on KITTI, backbone on BDD100K), combining them (both) is the only configuration that consistently maintains stable and highly competitive routing behavior across both domains.

lutions is highly dataset-dependent. On KITTI (Figure 8(a)), lacking spatial context entirely (1×1) yields the worst performance, whereas the denser 8×8 grid remains relatively competitive. In contrast, on BDD100K (Figure 8(b)), the 8×8 grid performs noticeably worse, likely because finer spatial representations are more sensitive to local feature noise in diverse scenes, while the 1×1 router performs much closer to the 2×2 design. These results suggest that a modest spatial structure (2×2) offers a reasonable balance between preserving spatial context and reducing sensitivity to local feature noise. Accordingly, we adopt the 2×2 grid as the default configuration.

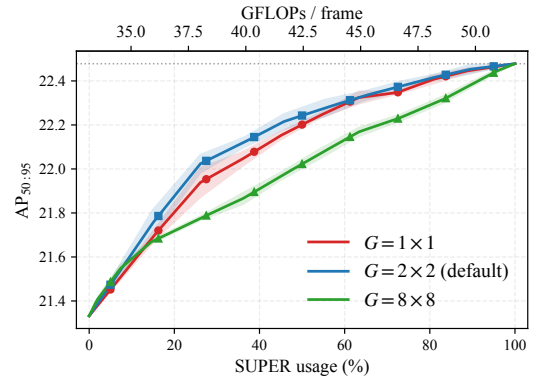
3) Path-Conditioned Sampling Ratio p

During training, the previous routing decision is simulated by sampling the preceding execution path, where the super path is selected with probability p . We evaluate three settings: $p \in \{0.0, 0.5, 1.0\}$.

Figure 9 shows that the balanced setting ($p = 0.5$) consistently achieves the most robust performance by reducing the train-deployment distribution gap. During deployment, the router receives features generated by whichever path was executed on the previous frame. Because the super and base



(a) KITTI



(b) BDD100K

FIGURE 8. Ablation on router grid resolution on (a) KITTI and (b) BDD100K. While the 1×1 and 8×8 configurations exhibit severe dataset-dependent instability, the 2×2 grid consistently maintains a robust and competitive accuracy-efficiency trade-off across both environments.

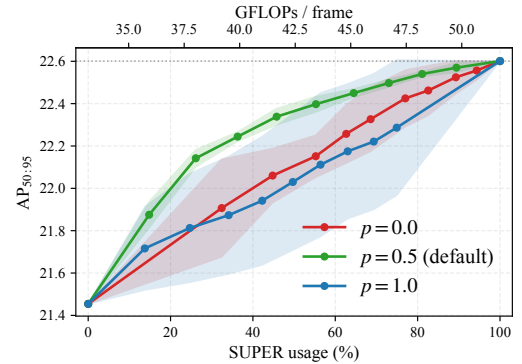


FIGURE 9. Ablation on the path-conditioned sampling ratio p on BDD100K. The balanced setting ($p = 0.5$) achieves the most robust performance by reducing train-deployment distribution shifts.

paths produce slightly different feature representations, training with only one path ($p = 0.0$ or 1.0) leaves the router vulnerable when the alternative path is encountered at runtime. In contrast, sampling both paths equally exposes the router to both feature distributions, ensuring more reliable advantage estimation. Table 5 further validates this by measuring the consistency of router predictions between the two execution paths for the same input frame. The balanced setting achieves

TABLE 5. Consistency of router predictions across different execution paths on the BDD100K validation set. MAE between $\hat{A}(F_{\text{super}})$ and $\hat{A}(F_{\text{base}})$ for the same frame, averaged over 5 seeds.

Training p	MAE
$p = 0.0$	1.16
$p = 0.5$ (default)	0.008
$p = 1.0$	1.95

a substantially lower MAE of 0.008 compared to the single-path baselines (1.16 and 1.95), confirming that exposure to both path-conditioned features enables highly stable routing decisions.

VI. CONCLUSION

In this paper, we proposed a feedback-driven dynamic-depth routing framework for budget-adaptive camera-based object detection in Advanced Driver Assistance Systems (ADAS). By decoupling the router's training phase from deployment constraints, we formulated frame-level execution-path selection as a continuous advantage regression problem. During deployment, a lightweight causal router eliminates inference latency by exploiting the temporal coherence of preceding frames, while a feedback controller dynamically adjusts the execution threshold to satisfy time-varying resource targets without retraining.

Evaluations demonstrated that our approach effectively balances the accuracy-efficiency Pareto frontier, reducing energy consumption and per-frame latency by 11% to 21% while maintaining average precision within 0.1 points of the full-capacity baseline. Furthermore, live closed-loop on-device evaluations confirmed the system's practical viability under continuous scene variations, tracking dynamic frame rate and energy budgets with a relative error of under 3%.

Overall, this framework provides a robust and flexible solution for resource-constrained edge platforms deployed in real-world ADAS applications. In future work, we will extend the current binary routing formulation to support multiple execution depths, enabling finer-grained computational scaling and more flexible runtime resource adaptation on edge devices.

REFERENCES

- [1] N. Venkateswaran, W. J. Hans, and N. Padmapriya, "Deep learning based robust forward collision warning system with range prediction," *Multimedia Tools and Applications*, vol. 80, no. 14, pp. 20 849–20 867, 2021.
- [2] C. Sun, Y. Li, H. Li, E. Xu, Y. Li, and W. Li, "Forward collision warning strategy based on millimeter-wave radar and visual fusion," *Sensors*, vol. 23, no. 23, p. 9295, 2023.
- [3] C. Chitraranjan, V. Vipulanathan, and T. Sritharan, "Vision-based collision warning systems with deep learning: A systematic review," *Journal of Imaging*, vol. 11, no. 2, p. 64, 2025.
- [4] B.-J. Kim and S.-B. Lee, "A study on the evaluation method of autonomous emergency vehicle braking for pedestrians test using monocular cameras," *Applied Sciences*, vol. 10, no. 13, p. 4683, 2020.
- [5] S. Rajendar, D. Rathinasamy, R. Pavithra, V. K. Kaliappan, and S. Gnana-murthy, "Prediction of stopping distance for autonomous emergency braking using stereo camera pedestrian detection," *Materials Today: Proceedings*, vol. 51, pp. 1224–1228, 2022.
- [6] K. Alawaji, R. Hedjar, and M. Zuair, "Traffic sign recognition using multi-task deep learning for self-driving vehicles," *Sensors*, vol. 24, no. 11, p. 3282, 2024.
- [7] C.-i. Kim, J. Park, Y. Park, W. Jung, and Y.-s. Lim, "Deep learning-based real-time traffic sign recognition system for urban environments," *Infrastructures*, vol. 8, no. 2, p. 20, 2023.
- [8] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019.
- [9] T. Benoit-Cattin, D. Velasco-Montero, and J. Fernández-Berni, "Impact of thermal throttling on long-term visual inference in a cpu-based edge device," *Electronics*, vol. 9, no. 12, p. 2106, 2020.
- [10] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "Yolov10: Real-time end-to-end object detection," *Advances in neural information processing systems*, vol. 37, pp. 107 984–108 011, 2024.
- [11] Y. Tian, Q. Ye, and D. Doermann, "YOLOv12: Attention-centric real-time object detectors," *arXiv preprint arXiv:2502.12524*, 2025.
- [12] X. Lin, J. Peng, Z. Gan, J. Zhu, and J. Liu, "Yolo-master: Moe-accelerated with specialized transformers for enhanced real-time detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 18 440–18 449.
- [13] R. Sapkota, R. H. Cheppally, A. Sharda, and M. Karkee, "Yolo26: Key architectural enhancements and performance benchmarking for real-time object detection," *arXiv preprint arXiv:2509.25164*, 2025.
- [14] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "Detrs beat yolos on real-time object detection," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 16 965–16 974.
- [15] Z. Lin, Y. Wang, J. Zhang, and X. Chu, "Dynamicdet: A unified dynamic architecture for object detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 6282–6291.
- [16] W. Kang and H. Lee, "Adaptive depth networks with skippable sub-paths," *Advances in Neural Information Processing Systems*, vol. 37, pp. 33 213–33 231, 2024.
- [17] D. Kuhse, H. Teper, S. Buschjäger *et al.*, "You only look once at anytime (AnytimeYOLO): Analysis and optimization of early-exits for object-detection," *arXiv preprint arXiv:2503.17497*, 2025.
- [18] W. Kang, H. Lee, and J. Lee, "Anydepth-detr/yolo: Any-depth object detection with a single network," *arXiv preprint arXiv:2605.09407*, 2026.
- [19] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," *Advances in neural information processing systems*, vol. 35, pp. 16 344–16 359, 2022.
- [20] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017. [Online]. Available: <https://openreview.net/forum?id=B1ckMDqlg>
- [21] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.
- [22] C. Riquelme, J. Puigcerver, B. Mustafa, M. Neumann, R. Jenatton, A. Susano Pinto, D. Keysers, and N. Houlsby, "Scaling vision with sparse mixture of experts," *Advances in Neural Information Processing Systems*, vol. 34, pp. 8583–8595, 2021.
- [23] S. Wang, C. Xia, F. Lv, and Y. Shi, "Rt-detr3: Real-time end-to-end object detection with hierarchical dense positive supervision," in *WACV*, 2025, pp. 1628–1636.
- [24] Z. Liao, Y. Zhao, X. Shan, Y. Yan, C. Liu, L. Lu, X. Ji, and J. Chen, "Rt-detr4: Painlessly furthering real-time object detection with vision foundation models," *arXiv preprint arXiv:2510.25257*, 2025.
- [25] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *European conference on computer vision*. Springer, 2020, pp. 213–229.
- [26] F. Li, H. Zhang, S. Liu, J. Guo, L. M. Ni, and L. Zhang, "Dn-detr: Accelerate detr training by introducing query denoising," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 13 619–13 627.
- [27] H. Zhang, F. Li, S. Liu, L. Zhang, H. Su, J. Zhu, L. M. Ni, and H.-Y. Shum, "Dino: Detr with improved denoising anchor boxes for end-to-end object detection," *arXiv preprint arXiv:2203.03605*, 2022.
- [28] S. Teerapittayanon, B. McDanel, and H. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," in *Proc. Int. Conf. Pattern Recognit. (ICPR)*, 2016, pp. 2464–2469.

- [29] M. Figurnov, M. D. Collins, Y. Zhu, L. Zhang, J. Huang, D. Vetrov, and R. Salakhutdinov, "Spatially adaptive computation time for residual networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 1039–1048.
- [30] G. Huang, D. Chen, T. Li, F. Wu, L. van der Maaten, and K. Weinberger, "Multi-scale dense networks for resource efficient image classification," in *International Conference on Learning Representations*, 2018. [Online]. Available: <https://openreview.net/forum?id=Hk2aImxAb>
- [31] L. Yang, Y. Han, X. Chen, S. Song, J. Dai, and G. Huang, "Resolution adaptive networks for efficient inference," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 2369–2378.
- [32] F. Ilhan, K.-H. Chow, S. Hu, T. Huang, S. Tekin, W. Wei, Y. Wu, M. Lee, R. Kompella, H. Latapie *et al.*, "Adaptive deep neural network inference optimization with eenet," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024, pp. 1373–1382.
- [33] X. Wang, F. Yu, Z.-Y. Dou, T. Darrell, and J. E. Gonzalez, "Skipnet: Learning dynamic routing in convolutional networks," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 409–424.
- [34] Z. Wu, T. Nagarajan, A. Kumar, S. Rennie, L. S. Davis, K. Grauman, and R. Feris, "Blockdrop: Dynamic inference paths in residual networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 8817–8826.
- [35] J. Yu, L. Yang, N. Xu, J. Yang, and T. Huang, "Slimmable neural networks," in *International Conference on Learning Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=H1gMCsAqY7>
- [36] J. Yu and T. S. Huang, "Universally slimmable networks and improved training techniques," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 1803–1811.
- [37] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, "Once for all: Train one network and specialize it for efficient deployment," in *International Conference on Learning Representations*, 2020.
- [38] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 2117–2125.
- [39] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path aggregation network for instance segmentation," in *2018 IEEE/CVF conference on computer vision and pattern recognition*. IEEE, 2018, pp. 8759–8768.
- [40] G. Ghiasi, T.-Y. Lin, and Q. V. Le, "Nas-fpn: Learning scalable feature pyramid architecture for object detection," in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2019, pp. 7029–7038.
- [41] M. Tan, R. Pang, and Q. V. Le, "Efficientdet: Scalable and efficient object detection," in *2020 IEEE/CVF conference on computer vision and pattern recognition (CVPR)*. IEEE, 2020, pp. 10 778–10 787.
- [42] L. Yang, Z. Zheng, J. Wang, S. Song, G. Huang, and F. Li, "AdaDet: An adaptive object detection system based on early-exit neural networks," *IEEE Trans. Cogn. Develop. Syst.*, vol. 16, no. 1, pp. 332–345, 2024.
- [43] J. Lee, H. Kim, Y. Sung-Jo, and Y. Choi, "Input-adaptive dynamic neural network for efficient object detection toward resource-constrained deployment," *Electronics*, vol. 15, no. 11, p. 2310, 2026.
- [44] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan, and T. Darrell, "Bdd100k: A diverse driving dataset for heterogeneous multi-task learning," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 2636–2645.
- [45] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 1251–1258.
- [46] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? the kitti vision benchmark suite," in *2012 IEEE conference on computer vision and pattern recognition*. IEEE, 2012, pp. 3354–3361.
- [47] P. Sun, H. Kretzschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine *et al.*, "Scalability in perception for autonomous driving: Waymo open dataset," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 2446–2454.
- [48] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [49] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft coco: Common objects in context," in *European conference on computer vision*. Springer, 2014, pp. 740–755.
- [50] NVIDIA, "NVIDIA TensorRT," <https://developer.nvidia.com/tensorrt>, 2025. [Online]. Available: <https://developer.nvidia.com/tensorrt>



HYUNGSEOP LEE received the B.S. degree in Embedded Systems Engineering from Incheon National University, Incheon, South Korea, in 2024, where he is currently pursuing the M.S. degree as a member of the Embedded AI Laboratory. His research interests include computer vision, object detection, efficient deep learning models, and adaptive inference for resource-constrained systems.



JIHO LEE received the B.S. degree in Embedded Systems Engineering from Incheon National University, Incheon, South Korea, in 2024, where he is currently pursuing the M.S. degree as a member of the Embedded AI Laboratory. His research interests include quantization-aware training for model compression of deep neural networks.



SUNWOO LEE is currently pursuing the B.S. degree in Embedded Systems Engineering at Incheon National University, Incheon, South Korea, where he is actively participating in research as an undergraduate member of the Embedded AI Laboratory. His research interests include computer vision.



WOOCUL KANG (Member, IEEE) received the Ph.D. degree in computer science from the University of Virginia, in 2009. He was a Senior Researcher with the Electronics and Telecommunications Research Institute, South Korea, from 2000 to 2004 and from 2009 to 2012, and a Postdoctoral Research Associate with the University of Illinois at Urbana-Champaign, USA, from 2012 to 2013. Since 2013, he has been a Professor with the Department of Embedded Systems Engineering, Incheon National University, Incheon, South Korea. His research interests include deep learning for edge devices, real-time systems, distributed middleware, and feedback control of computing systems.

...